AI SHOPPING AGENT

Autonomous Market Intelligence & Price Comparison System

Stack: FastAPI · Groq LLaMA-3 · Gemini 2.5 Flash Lite · MongoDB Atlas · MCP Browser Client

Project Report | Prepared by: Nayana | Prepared for: Engineering Management | Date: June 2026

1. Project Overview

The AI Shopping Agent is a fully autonomous, multi-agent web application that helps Indian consumers determine whether to buy or wait on a product based on real-time market pricing and historical price intelligence. The user pastes a product URL from any supported Indian e-commerce platform; the system autonomously scrapes the page, queries competitor prices across the market, consults a persistent MongoDB price history database, and delivers a clear BUY NOW or WAIT recommendation — all within a single browser interaction.

The intelligence lives in a ReAct-style agentic loop where LLaMA-3 on Groq decides which tools to call and in what order, while Google Gemini with live Search grounding handles competitor discovery. The system has been operational since March 2026 and has accumulated over 505 real price records across 22 products.

1.1 Problem Statement & Solution

The Problem: The Indian e-commerce market is highly fragmented, with prices varying by 15–30% across platforms like Amazon, Flipkart, Croma, and D2C brand stores due to volatile algorithmic pricing and localized promotions. Manually checking up to a dozen sites to compare deals is tedious, time-consuming, and creates severe discovery fatigue. Furthermore, retailers frequently exploit the "Discount Illusion" by inflating base prices right before a sale to display deceptive price drops. Without historical context, shoppers cannot tell a genuine deal from a fake one. Programmatically automating this comparison is blocked by aggressive anti-bot perimeters (Akamai, Cloudflare, CAPTCHAs) that instantly block standard scrapers with `403 Forbidden` errors.

The Solution: This system solves these bottlenecks through a coordinated, multi-agent automated architecture:

- **Bypassing Anti-Bot Barriers:** The backend isolates network operations by spawning an independent Node.js subprocess using the Model Context Protocol (`mcp-server-fetch-typescript`) routed through residential proxies via ScraperAPI to effortlessly circumvent CAPTCHAs and fetch raw data.
- **Aggregating Fragmented Prices:** A three-tier extraction engine combines robust JSON-LD schema parsing, precise CSS selectors, and a fallback LLaMA-3 processing loop on Groq to clean and extract product details seamlessly.
- **Validating Real Deals:** The system passes the product to Gemini 2.5 Flash Lite with live Google Search Grounding to automatically discover competitor prices. These are parsed through a median filter to remove accessory noise and verified against a persistent MongoDB Atlas historical baseline to deliver a definitive, statistically backed **BUY NOW** or **WAIT** recommendation instantly.

1.2 Target Platforms

The system has native HTML parsers for Amazon and Flipkart, and a JSON-LD fallback for all other retailers. Competitor search covers Amazon, Flipkart, Croma, Reliance Digital, Vijay Sales, Tata CLiQ, Myntra, Nykaa, JioMart, Headphone Zone, and official brand stores.

2. Project Structure

The repository separates concerns cleanly. Backend logic, agent skills, frontend assets, utility scripts, and tests each live in their own folder. Nothing from the agent layer bleeds into the server, and nothing from the server touches the frontend directly.

Path	Purpose
`backend/server.py`	FastAPI app — MCP spawner, cache layer, REST API
`backend/db.py`	MongoDB schemas, price history, URL cache
`backend/api_balancer.py`	Gemini key rotation & Groq client factory
`skills/shopping_agent.py`	Autonomous ReAct loop — core agent brain
`skills/scraper_skill.py`	HTML fetcher, native parsers & LLM fallbacks
`frontend/index.html + styles.css + app.js`	Dashboard UI, step-progress loader, Chart.js rendering
`scripts/seed_db.py`	Seeding utility — pre-populates price history for testing
`scripts/view_db.py`	Generates price_history_report.md from live DB
`scripts/clear_cache.py`	Drops url_cache — forces fresh fetch on next run
`scripts/list_models.py`	Lists available Groq models for the configured API key
`tests/`	Unit, integration, and crash-recovery test scripts
`.agents/skills/shopping_agent/SKILL.md`	AI-agent skill manifest (machine-readable)
`.env`	API keys & MongoDB URI — excluded from version control

3. System Architecture

The system is built in four layers. Requests enter through the FastAPI server, which handles caching and spawns the MCP browser client on a cache miss. The agent brain (LLaMA-3 on Groq) runs the ReAct loop, calling tools that delegate to the scraping engine, Gemini search, and MongoDB. Results return to the browser as structured JSON rendered by the frontend.

3.1 Layer Overview

Layer	Components	Responsibilities
Presentation	index.html, styles.css, app.js	URL input, animated progress loader, price cards, Chart.js history graph
API Gateway	backend/server.py (FastAPI)	URL normalisation, store detection, cache lookup, MCP session lifecycle
Intelligence	shopping_agent.py, scraper_skill.py, api_balancer.py	ReAct loop, HTML scraping, AI extraction, competitor search, statistical filtering
Data	backend/db.py (MongoDB Atlas)	Price history persistence, URL result cache with 1-hour TTL

3.2 Request Flow

A complete request follows this path from URL submission to rendered recommendation:

Request Flow — URL Submission to Rendered Recommendation
Frontend — User pastes product URL
↓ POST /api/analyze
FastAPI Server — URL normalisation + store detection
↓ Check url_cache in MongoDB ◇ Cache HIT → Return instantly (< 1 hour TTL)
Cache MISS → Spawn MCP Browser Client (mcp-server-fetch-typescript)
↓ ScraperAPI proxy fetch (render=false, ~3 s)
Groq / LLaMA-3.3-70B Agent — ReAct loop
↓ Tool 1: fetch_product_data → JSON-LD → CSS Selectors → Groq LLM fallback
↓ Tool 2: search_competitors → Gemini + Google Search → Median filter + deduplication
↓ Save to price_history → Fetch historical baselines from MongoDB
↓ Tool 3: final_recommendation → Save to url_cache → JSON to browser
Frontend renders: price cards + BUY NOW / WAIT badge + Chart.js history graph

4. Technology Stack

Category	Technology	Purpose
Agent Reasoning	Groq / LLaMA-3.3-70B	ReAct loop orchestration, HTML/JSON-LD data extraction
Competitor Search	Gemini 2.5 Flash Lite + Google Search	Live price discovery with real-time search grounding
Web Scraping	ScraperAPI + MCP (mcp-server-fetch-typescript)	CAPTCHA bypass, raw HTML via headless Node.js subprocess
HTML Parsing	BeautifulSoup4 + JSON-LD	Deterministic CSS selector & structured data extraction
Backend Framework	FastAPI + Uvicorn	Async REST API, static file serving
Database	MongoDB Atlas (PyMongo)	Price history logs, URL result cache with TTL
Frontend	HTML5 / CSS3 / Vanilla JS	Dashboard UI with glassmorphic design and CSS animations
Charting	Chart.js	Historical price deviation metrics visualisation
Key Management	api_balancer.py	Random selection across up to 10 Gemini API keys for quota distribution
CLI Search	DuckDuckGo DDGS	Headless mode URL discovery with direct search URL fallback

5. Logical Workflows

This section describes the three major processing workflows inside every analysis request. Each corresponds to a distinct phase of the agent loop and has its own failure-handling strategy.

5.1 Secure Fetching Layer

The backend does not make HTTP requests directly. It launches an isolated Node.js process using the Model Context Protocol tool mcp-server-fetch-typescript, communicating with it over stdio. This subprocess is the only part of the system that touches the external internet for scraping, and it does so without access to the Python server's environment variables or in-memory data.

The actual fetch goes through ScraperAPI, which provides rotating residential proxies and bot-detection circumvention. JavaScript rendering is disabled (render=false) to keep fetch latency around 3 seconds rather than 15. The Python server passes a single URL to the MCP tool and receives raw HTML back. Nothing else crosses the process boundary.

5.2 Multi-Tiered Extraction Engine

E-commerce sites change their HTML layout regularly. A system that relies on a single parsing strategy breaks silently when a retailer redesigns their product page. To guard against this, the extraction engine has three independent layers tried in sequence:

Tier 1 — JSON-LD Schema Extractor

BeautifulSoup scans every <script type="application/ld+json"> tag on the page. Most legitimate retailers embed structured Product schema data here for search engine consumption. It is far more stable than visual HTML layout. When found, the clean JSON is sent to Groq for parsing. This is the fastest and most reliable tier.

Tier 2 — Native CSS Selector Parsers

For Amazon and Flipkart specifically, the engine uses hardcoded CSS selectors targeting known price and title elements (span#productTitle and span.a-price-whole for Amazon; div.Nx9bqj for Flipkart). These run in milliseconds when they hit and fall through gracefully if the selector returns nothing.

Tier 3 — Groq LLM Parser

If neither of the above yields usable data, the HTML is stripped of all script and style tags, truncated to 100,000 characters, and sent to LLaMA-3 with a prompt instructing it to identify the main product name and current sale price. This tier is slower but acts as a reliable backstop for unusual page layouts or retailers outside the top two. The three tiers together mean the system degrades gracefully — a retailer redesign may break Tier 2 but Tier 1 or Tier 3 will still produce a usable result.

5.3 Competitor Discovery and Historical Analysis

Once primary product details are extracted, the agent calls search_competitors. This delegates to Gemini 2.5 Flash Lite with Google Search grounding, allowing it to issue live Google queries and incorporate real-time results.

Why Raw Titles Cannot Be Searched Directly

Product titles from e-commerce pages are aggressively SEO-optimised. A Flipkart title like 'Noise Twist Go Round Dial Smartwatch with BT Calling 1.39" Display...' would return zero cross-retailer matches if searched verbatim. The Gemini prompt explicitly instructs the model to first derive a clean core product entity (Brand + Model + Key Spec) before constructing any search query.

Outlier Filtering via Median

Search results routinely include accessories priced at a fraction of the main product. After Gemini returns results, a Python median filter discards any result priced below 30% of the median. A hardcoded blacklist also removes price aggregator sites (Smartprix, MySmartPrice, 91mobiles, Gadgets360, Buyhatke) and news outlets that appear in search results but do not sell directly.

Historical Price Baselines

After competitor prices are collected and filtered, they are saved to MongoDB's price_history collection with a timestamp. The system then queries that collection for all historical records matching the product name and computes three statistics:

- All-time lowest price — the floor used to judge whether the current price is a genuine deal.
- All-time highest price — context for how inflated prices can reach for this product.
- Average price across all records — the baseline against which the current best price is compared.

Decision rule: If the cheapest current deal is at or below the all-time low (within 5%), the action is BUY NOW. If it exceeds the historical average, the action is WAIT. For new products with no history, the agent compares across stores and recommends buying from whichever is cheapest.

6. Module Breakdown

backend/server.py

The FastAPI application. Exposes POST /api/analyze and serves frontend static files. Handles URL normalisation (strips Amazon /ref= tracking paths, UTM parameters, and fragments to produce a canonical cache key), store name detection from the URL domain, cache checking, MCP subprocess lifecycle, and writing results to cache on a successful run.

backend/db.py

All MongoDB operations in one place. get_db_collection() returns a collection handle from MONGO_URI. save_extracted_prices() cleans price strings to floats and bulk-inserts records. get_price_history() runs a case-insensitive product name query and returns min/max/average statistics. check_url_cache() and save_to_url_cache() manage the 1-hour result cache. If MONGO_URI is not set, the system operates in stateless mode — history is skipped and caching is disabled.

backend/api_balancer.py

get_gemini_client() reads up to ten Gemini API keys (GEMINI_API_KEY through GEMINI_API_KEY_9) and returns a client initialised with a randomly selected key, providing basic quota spreading across keys. get_groq_client() returns an AsyncGroq instance, force-reloading .env on each call so new keys take effect without a server restart.

skills/shopping_agent.py

The agent brain. Defines the three Groq tool schemas, the system prompt with decision guidelines, and the message accumulation loop. The messages list grows with every turn and is re-sent to Groq on each iteration to maintain context across stateless API calls. find_cheapest_deal() strips currency characters and finds the numerically lowest price across all collected results. A 10-iteration failsafe prevents runaway loops. A safeguard check also prevents final_recommendation from being called before search_competitors, ensuring recommendations are always grounded in real market data.

skills/scraper_skill.py

Two async functions. fetch_html_via_mcp() calls the MCP server's get_raw_text tool with a ScraperAPI-proxied URL and returns raw HTML. extract_data_with_groq() runs the three-tier extraction strategy and returns a dict with store, title, and price keys.

frontend/

A self-contained single-page application with no build step. index.html defines the layout including an animated four-step progress loader, a price comparison card grid, and a Chart.js canvas. styles.css implements a glassmorphic dark aesthetic with CSS custom properties and keyframe animations. app.js manages the fetch call, animates the loading steps, and renders all result data client-side.

7. Live Database Status

The MongoDB Atlas database has been actively in use since March 2026 and contains 505 price history records across 22 distinct products, along with 3 cached URL analysis results.

Product	All-Time Low	All-Time High	Avg Price	Records
Apple AirPods Pro 2nd Gen	₹21,559	₹26,624	₹24,052	120
Apple iPhone 16 Pro 128 GB	₹1,07,259	₹1,28,204	₹1,18,640	120
Noise Twist Go Smartwatch	₹1,399	₹4,999	₹1,879	67
Sony MDR-EX15AP Headphones	₹13	₹890	₹701	43
Samsung Galaxy S25 5G 256 GB	₹56,999	₹79,999	₹71,131	19
Apple iPhone 17 Pro (Cosmic Orange)	₹1,22,900	₹1,34,900	₹1,31,218	8
HP Victus i5 RTX 3050 Laptop	₹71,990	₹74,990	₹74,240	4
Realme Buds T310 ANC Earbuds	₹2,099	₹2,399	₹2,235	11
JBL C100SI Wired Headphones	₹599	₹699	₹649	12
Boat Airdopes 212 TWS Earbuds	₹849	₹1,149	₹1,011	17

The AirPods Pro 2nd Gen and iPhone 16 Pro are the most comprehensively tracked products with 120 records each, providing statistically robust baselines. Products like the iPhone 17 Pro (8 records) are newer additions whose baselines will strengthen over time as more searches accumulate.

8. Key Design Decisions

Groq for Agent Reasoning, Gemini for Search

The two models are used for what each does best. Groq's inference latency on LLaMA-3 is low enough that multi-turn agentic loops remain responsive. But LLaMA-3 has no native search grounding. Gemini's Google Search grounding is what makes live competitor discovery possible without maintaining a custom scraper per retailer. The multi-key rotation in api_balancer.py exists because Gemini free-tier quota is the primary bottleneck under load.

MCP over Direct HTTP

Amazon and Flipkart return CAPTCHAs or structurally empty pages to non-browser HTTP clients. The MCP subprocess + ScraperAPI routes fetches through rotating residential proxies with a real browser fingerprint. The MCP abstraction also keeps the agent's tool call simple: it calls get_raw_text with a URL and receives HTML. All proxy configuration and session management lives in the subprocess.

Tiered Extraction over Pure LLM

Sending raw HTML to a language model works but costs tokens and can hallucinate prices. The native CSS selectors for Amazon and Flipkart are fast and precise. JSON-LD parsing targets stable structured data rather than visual layout. LLM extraction is reserved for cases where provisional parsing genuinely cannot help, keeping the common case cheap and reliable.

Median Filter for Accessory Exclusion

The 30%-of-median threshold was chosen empirically. It drops clear outliers — accessories priced at ₹12 for a ₹7,000 product — without affecting legitimate price variation between retailers, which rarely exceeds 25–30% for the same product on the same day.

1-Hour URL Cache

A full analysis takes 15–30 seconds on a cold run and consumes API credits from three services. Caching the complete result for one hour makes repeat lookups instant and keeps demo sessions practical. The normalised cache key ensures that the same product reached via different affiliate URLs resolves to a single cache entry.

9. Testing

The tests/ directory covers individual components and end-to-end paths. All tests use Python's standard unittest and asyncio modules.

Test File	What It Covers
`test_airpods.py`	End-to-end regression test: AirPods search, extraction, and recommendation
`test_500.py`	HTTP 500 error handling and graceful frontend error messaging
`test_apify_sse.py`	Server-Sent Events streaming prototype (early architecture exploration)
`test_crash.py`	Stress test for edge cases — validates 10-iteration failsafe and exception propagation
`test_db.py`	MongoDB connection, insert, and query operations
`test_fetch_tools.py`	MCP fetch tool availability and response format
`test_flipkart.py`	Flipkart-specific CSS selector parsing
`test_gemini_search.py`	Gemini competitor search grounding and result filtering
`test_groq.py`	Groq LLM extraction accuracy and JSON output format
`test_grounding.py`	Google Search grounding integration
`test_keys.py`	API key loading and balancer rotation logic
`test_mcp_amazon.py`	Amazon MCP fetch and price extraction pipeline
`test_mcp_search.py`	MCP-based search result handling
`test_scrape.py`	Full scraper skill integration test
`test_search.py`	DuckDuckGo CLI search module

The debug/ directory contains diagnostic artefacts from active development: a screenshot of Amazon's bot-detection interception (amazon_bot_diagnosis.png), raw Gemini search output (gemini_search_test.txt), Groq API response dumps (test_groq_output.txt), and the generated price history report. The search_out.txt and search_debug.txt files are written to disk at runtime, providing a persistent audit trail of each analysis run.

10. Setup and Deployment

Prerequisites

- Python 3.11+ and Node.js 18+ installed on the host
- MongoDB Atlas cluster with a connection URI
- Groq API key (free tier sufficient for moderate usage)
- One or more Gemini API keys (free-tier quota is the main constraint under load)
- ScraperAPI key (free tier: 1,000 credits/month)

Installation

Install Python dependencies:

```
pip install -r requirements.txt
```

The MCP Node.js server installs automatically via npx on first request — no manual Node setup needed.

Create a .env file in the project root:

```
GROQ_API_KEY=your_key
GEMINI_API_KEY=your_key
GEMINI_API_KEY_1=optional_second_key
MONGO_URI=your_mongodb_atlas_connection_string
```

Optionally seed historical data for testing the recommendation engine:

```
python scripts/seed_db.py
```

Start the server:

```
uvicorn backend.server:app --reload --port 8000
```

Open http://localhost:8000, paste any Indian e-commerce product URL, and click Analyze Market.

Utility Scripts

- **`scripts/seed_db.py`** — Pre-populates MongoDB with baseline price records for testing
- **`scripts/clear_cache.py`** — Purges the url_cache collection, forcing fresh analysis on next request
- **`scripts/view_db.py`** — Generates price_history_report.md showing all stored pricing records
- **`scripts/list_models.py`** — Lists available Groq models for the configured API key

11. Known Limitations

- **Gemini quota exhaustion:** The competitor search step consumes one Search Grounding call per analysis. On the free tier this is exhausted quickly under load. The multi-key balancer mitigates but does not eliminate it.
- **Product name fragmentation:** The same physical product can appear under slightly different title strings in MongoDB (e.g., 'Apple AirPods 4' vs 'Apple AirPods 4 MXP63HNA'). This fragments historical data and reduces statistical value. A normalisation step at write time would resolve this.

- **Cold-start history:** On the first run for any product, MongoDB has no records. Historical baselines only become meaningful after several days of repeated queries for the same product.

13. Conclusion

The AI Shopping Agent successfully demonstrates a practical application of autonomous multi-agent AI for a consumer-facing problem. By combining the reasoning strength of LLaMA 3.3-70B with the real-time search grounding of Google Gemini, and grounding both in a growing MongoDB historical database, the platform delivers actionable BUY NOW or WAIT recommendations that are statistically defensible rather than heuristic guesses.

The project has matured from an initial CLI prototype to a full-stack web application with a caching layer, a robust three-tier scraping pipeline, and over 500 real price records spanning three months of live operation. The architecture is cleanly layered, making individual components straightforward to test, update, or replace independently.

The primary area requiring attention before a broader rollout is product name normalisation in the database to combine duplicate product names under unified baselines. This low-complexity fix would meaningfully improve data quality and operational breadth across the system.

Overall, the project is in a healthy operational state and demonstrates strong potential for expansion into a production-grade price intelligence service for the Indian e-commerce market.

— End of Report — | AI Shopping Agent Project Report | June 2026 | Prepared by: Nayana